

DSA STUDY BLUEPRINT SERIES

# 94. Min Distance between BST Nodes

Finding Minimum Absolute Difference in BSTs

Section 09 • C++ Core Data Structures & Algorithms

# Core Concepts & Visual Blueprint

Theoretical models and memory architectures

---

## Key Principles

- **Inorder Property:** Inorder traversal yields sorted elements.
- Track difference between current node and previous node.

## Memory & Execution Blueprint

Sorted Inorder: [1, 3, 6]  $\Rightarrow$  Min Diff =  $3 - 1 = 2$

# C++ Implementation Reference

Standard code layout and syntax

---

```
int minDiff = 1e9, prevVal = -1;
void getMinDiff(TreeNode* root) {
    if (!root) return;
    getMinDiff(root->left);
    if (prevVal != -1) minDiff = min(minDiff, root->val - prevVal);
    prevVal = root->val;
    getMinDiff(root->right);
}
```

# Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

## Performance Table

| Aspect / Case    | Complexity |
|------------------|------------|
| Time Complexity  | $O(N)$     |
| Space Complexity | $O(H)$     |

## Key Practices & Gotchas

- **Important:** Yields  $O(N)$  time by running inorder traversal in a single pass.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

# Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

## Execution Walkthrough

Let's trace BST constraint validation checks verifying node ranges `[Min, Max]`:

| Active Node | Valid range limit    | Comparison check                | Recursive branch step                         |
|-------------|----------------------|---------------------------------|-----------------------------------------------|
| 15 (Root)   | $[-\infty, +\infty]$ | True $(-\infty < 15 < +\infty)$ | Recurse Left child with range $[-\infty, 15]$ |
| 10 (Left)   | $[-\infty, 15]$      | True $(-\infty < 10 < 15)$      | Recurse Right child with range $[10, 15]$     |

# Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

---

## Key Variations & Follow-up Questions

- **Inorder property:** BST Inorder traversals yield strictly increasing sorted lists.
- **Tree Balancing:** Balanced tree heights (AVL/Red-black) guarantee logarithmic search times.
- **Related LeetCode Problems:** #98 Validate Binary Search Tree, #700 Search in a Binary Search Tree, #108 Convert Sorted Array to Binary Search Tree.